K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 3

Unit Title: Sorting Algorithms

Assignment Title: Sorting Performer Analyzer

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Shreya Verma

Roll Number: 2501730173

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

INTRODUCTION

This assignment focuses on implementing and analyzing three sorting algorithms: **Insertion Sort, Merge Sort, and Quick Sort**. The objective is to compare their performance on different types of datasets and relate the observed results with their theoretical time complexities.

Algorithms Implemented

1. INSERTION SORT

- Time Complexity:
  - Best Case: $O(n)$
  - Worst Case: $O(n^2)$
- Stable: Yes
- In-place: Yes

Insertion Sort works by inserting each element into its correct position in the sorted portion of the array.

2. MERGE SORT

- Time Complexity: $O(n \log n)$
- Stable: Yes
- In-place: No

  Merge Sort uses the divide-and-conquer approach. It divides the array into halves, sorts them recursively, and then merges them.

3. QUICK SORT

- Time Complexity:
  - Average Case: $O(n \log n)$
  - Worst Case: $O(n^2)$
- Stable: No
- In-place: Yes

  Quick Sort partitions the array around a pivot. A randomized pivot was used to avoid worst-case performance.

Dataset Description

The following datasets were generated:

- **Random Data**
- **Sorted Data**
- **Reverse Sorted Data**

Each dataset was tested with sizes:

- 1000
- 5000
- 10000

Execution Time Results

Size	Type	Insertion	Merge	Quick
1000	random	71.07	3.47	3.24
1000	sorted	0.60	2.22	3.48
1000	reverse	103.97	2.51	2.23
5000	random	1480.19	31.95	15.57
5000	sorted	0.97	24.28	24.11
5000	reverse	2717.63	13.73	31.47
10000	random	6644.5	79.35	43.51
10000	sorted	1.85	28.02	60.56
10000	reverse	13210.08	45.66	51.57

Analysis

Random Data

- Merge Sort and Quick Sort performed efficiently.
- Insertion Sort was slower due to $O(n^2)$ complexity.

Sorted Data

- Insertion Sort performed very fast (best case).
- Quick Sort showed slower performance due to poor pivot selection in worst-case scenarios.
- Merge Sort remained consistent.

Reverse Sorted Data

- Insertion Sort performed worst.
- Quick Sort performance degraded but improved with randomized pivot.
- Merge Sort remained stable and consistent.

Theory vs Practical Observation

- Algorithms with **$O(n^2)$** complexity (Insertion Sort) became very slow as input size increased.
- Algorithms with **O(n log n)** complexity (Merge Sort and Quick Sort) scaled much better.
- Experimental results closely matched theoretical expectations.

Quick Sort Special Observation

Quick Sort can perform poorly ($O(n^2)$) when the pivot selection is not optimal, especially for sorted or reverse-sorted data.

To overcome this, a **randomized pivot strategy** was used, which improved performance and avoided deep recursion.

Stability and Memory Comparison

Algorithm	Stable	In-place
Insertion Sort	Yes	Yes
Merge Sort	Yes	No
Quick Sort	No	Yes

Conclusion

- Merge Sort is reliable and consistent for all types of data.
- Quick Sort is fast but depends on pivot selection.
- Insertion Sort is efficient only for small or nearly sorted datasets.

References

- Course notes (Data Structures – Unit 3)
- GeeksforGeeks (Sorting Algorithms)
- Python documentation (time module)